

02_alert-html

#html #css #javascript #frontend #api #SIEM #SOC

1 Ubicación del archivo dentro del proyecto

```
siem-lab/  
└─ frontend/  
    └─ alert.html
```

El archivo `alert.html` se encuentra en la raíz de la carpeta del frontend:

```
frontend/
```

Este archivo define la página de detalle de una alerta concreta.

Su función principal es mostrar los datos básicos de una alerta y permitir modificar su estado mediante tres acciones:

```
ACK  
CLOSE  
REOPEN
```

Es decir, esta página representa la parte de gestión individual de alertas dentro del flujo SOC básico del laboratorio.

2 Comando utilizado para visualizar el archivo

```
cd ~/siem-lab  
sed -n '1,320p' frontend/alert.html
```

Desglose del comando:

```
cd ~/siem-lab
```

Sitúa la terminal en la raíz del proyecto.

```
sed
```

Ejecuta el programa `sed`, utilizado para visualizar contenido de archivos.

```
-n
```

Evita imprimir todo el archivo automáticamente.

```
'1,320p'
```

Indica que se impriman las líneas de la 1 a la 320.

```
frontend/alert.html
```

Es la ruta del archivo que se quiere visualizar.

3 Código completo del archivo

```
<!doctype html>  
<html lang="es">  
<head>  
  <meta charset="utf-8" />  
  <meta name="viewport" content="width=device-width,initial-scale=1" />
```

```

<title>SIEM Lab – Detalle de alerta</title>
<link rel="stylesheet" href="./assets/styles.css" />
</head>
<body>
  <header class="topbar">
    <div class="container">
      <div class="brand">
        <span class="logo">SIEM</span>
      </div>
      <h1>Detalle de alerta</h1>
      <p class="muted">Contexto y gestión de estado</p>
    </div>

    <div class="actions">
      <a class="btn" href="./index.html">← Volver</a>
    </div>
  </header>

  <main class="container">
    <section class="card">
      <div class="card-header row">
        <div>
          <h2 id="title">Alerta #</h2>
          <p id="subtitle" class="muted"></p>
        </div>

        <div class="actions">
          <button id="btnAck" class="btn primary" type="button">ACK</button>
          <button id="btnClose" class="btn danger" type="button">CLOSE</button>
          <button id="btnReopen" class="btn" type="button">REOPEN</button>
        </div>
      </div>

      <div id="errorBox" class="alert error hidden"></div>
      <div id="infoBox" class="alert info hidden"></div>

      <div class="grid">
        <div class="kv">
          <div class="k">id</div><div class="v" id="v_id"></div>
          <div class="k">status</div><div class="v"><span id="v_status" class="badge"></span></div>
          <div class="k">group_key</div><div class="v" id="v_group_key"></div>
          <div class="k">rule_id</div><div class="v" id="v_rule_id"></div>
          <div class="k">event_id</div><div class="v" id="v_event_id"></div>
          <div class="k">created_at</div><div class="v" id="v_created_at"></div>
          <div class="k">updated_at</div><div class="v" id="v_updated_at"></div>
        </div>

        <div class="panel">
          <h3>Título</h3>
          <p id="v_title" class="mono"></p>

          <h3>Notas</h3>
          <ul class="muted">
            <li>Este MVP gestiona solo el estado (<code>open|ack|closed</code>).</li>
            <li>Los eventos asociados (detalle del evento) no están en <code>AlertOut</code>; se
integrarían usando <code>/events</code> en una iteración posterior.</li>
          </ul>
        </div>
      </section>
    </main>

    <script src="./assets/app.js"></script>

```

```
<script src="./assets/alert_detail.js"></script>
</body>
</html>
```

4 Función general del archivo

El archivo `alert.html` define la página de detalle de una alerta.

Esta página se abre desde el listado principal mediante una URL con parámetro `id`.

Ejemplo:

```
http://localhost:5173/alert.html?id=1
```

El archivo HTML no carga la alerta por sí solo. Solo define la estructura visual.

La lógica real está en:

```
frontend/assets/alert_detail.js
```

La relación es:

```
alert.html
  ↓
define los elementos visuales

app.js
  ↓
define funciones comunes

alert_detail.js
  ↓
lee el id, consulta la API y actualiza el DOM
```

Desde esta vista el usuario puede:

- Ver el ID de la alerta.
- Ver el estado actual.
- Ver el `group_key`.
- Ver `rule_id` y `event_id`.
- Ver fechas de creación y actualización.
- Ver el título de la alerta.
- Cambiar el estado a `ack`.
- Cambiar el estado a `closed`.
- Reabrir la alerta como `open`.

5 Estructura general del archivo

El archivo puede dividirse en cinco bloques:

```
<!doctype html>
<html lang="es">
```

Declaración del documento HTML.

```
<head>
...
</head>
```

Metadatos, título y enlace al CSS.

```
<header class="topbar">
  ...
</header>
```

Cabecera superior con el título de la página y enlace de vuelta.

```
<main class="container">
  ...
</main>
```

Contenido principal con la tarjeta de detalle.

```
<script src="./assets/app.js"></script>
<script src="./assets/alert_detail.js"></script>
```

Carga de scripts JavaScript.

Visualmente:

```
alert.html
├─ doctype
├─ html lang="es"
├─ head
│   ├── charset
│   ├── viewport
│   ├── title
│   └─ styles.css
├─ body
│   ├── header.topbar
│   │   ├── logo
│   │   ├── título
│   │   └─ enlace Volver
│   ├── main.container
│   │   └─ section.card
│   │       ├── cabecera de alerta
│   │       ├── botones ACK/CLOSE/REOPEN
│   │       ├── errorBox
│   │       ├── infoBox
│   │       ├── bloque key-value
│   │       └─ panel de título/notas
│   └─ scripts
│       ├── app.js
│       └─ alert_detail.js
```

6 Análisis línea por línea

Declaración del tipo de documento

```
<!doctype html>
```

Indica que el documento usa HTML5.

Permite que el navegador interprete la página en modo estándar.

Apertura del documento HTML

```
<html lang="es">
```

Abre el documento HTML.

El atributo:

```
lang="es"
```

indica que el idioma principal de la página es español.

Esto ayuda a la accesibilidad, lectores de pantalla y motores de búsqueda.

Apertura del `head`

```
<head>
```

Inicia la sección de metadatos del documento.

El contenido de `head` no se muestra directamente dentro de la página, pero configura aspectos importantes del documento.

Codificación de caracteres

```
<meta charset="utf-8" />
```

Define la codificación como UTF-8.

Esto permite mostrar correctamente caracteres especiales, tildes, símbolos y textos en español.

Configuración responsive

```
<meta name="viewport" content="width=device-width,initial-scale=1" />
```

Configura la escala y el ancho de la página en dispositivos móviles.

Desglose:

```
width=device-width
```

Hace que el ancho se ajuste al ancho real del dispositivo.

```
initial-scale=1
```

Establece el zoom inicial en 1.

Título de la pestaña

```
<title>SIEM Lab – Detalle de alerta</title>
```

Define el título de la pestaña del navegador.

En este caso indica que se trata de la vista de detalle de una alerta.

Enlace al CSS

```
<link rel="stylesheet" href="./assets/styles.css" />
```

Carga la hoja de estilos común del frontend.

Relación:

```
alert.html  
↓
```

```
assets/styles.css
```

Sin este archivo, la página tendría estructura HTML, pero no tendría el diseño visual del laboratorio.

Cierre del `head`

```
</head>
```

Cierra la sección de metadatos.

Apertura del `body`

```
<body>
```

Inicia el contenido visible de la página.

Todo lo que el usuario ve está dentro de `body`.

Cabecera superior

```
<header class="topbar">
```

Define la cabecera superior de la página.

La clase:

```
class="topbar"
```

aplica el estilo visual de barra superior definido en CSS.

Contenedor de cabecera

```
<div class="container">
```

Agrupar el contenido de la cabecera dentro de un ancho máximo.

La clase `container` centra el contenido y evita que ocupe toda la pantalla en monitores grandes.

Bloque de marca

```
<div class="brand">
```

Agrupar el logo y los textos principales de la cabecera.

Logo textual

```
<span class="logo">SIEM</span>
```

Muestra un logo textual con la palabra `SIEM`.

La clase `logo` aplica el estilo visual de recuadro con gradiente.

Contenedor de título

```
<div>
```

Contenedor simple para agrupar el título principal y el subtítulo.

Título principal

```
<h1>Detalle de alerta</h1>
```

Define el título principal visible de la página.
Indica que esta vista está centrada en una alerta individual.

Subtítulo

```
<p class="muted">Contexto y gestión de estado</p>
```

Muestra una descripción breve.
La clase muted aplica color secundario.
El texto indica claramente el objetivo de esta vista:

```
ver contexto  
gestionar estado
```

Cierre del bloque de marca

```
</div>  
</div>
```

Cierra el contenedor del título y el bloque .brand .

Contenedor de acciones

```
<div class="actions">
```

Agrupar acciones disponibles en la cabecera.
En esta página contiene un enlace para volver al listado.

Enlace volver

```
<a class="btn" href="./index.html">← Volver</a>
```

Crea un enlace con aspecto de botón.
Desglose:

```
class="btn"
```

Aplica estilo visual de botón.

```
href="./index.html"
```

Indica que al hacer clic se vuelve a la página principal.

```
← Volver
```

Texto visible del enlace.

Este enlace conecta:

```
alert.html
↓
index.html
```

Cierre de cabecera

```
</div>
</div>
</header>
```

Cierra acciones, contenedor y cabecera.

Apertura del contenido principal

```
<main class="container">
```

Define el contenido principal de la página.

La clase `container` mantiene alineación y ancho coherentes con la cabecera.

Tarjeta de detalle

```
<section class="card">
```

Crea una sección visual con estilo de tarjeta.

Toda la información de la alerta se agrupa dentro de esta tarjeta.

Cabecera de la tarjeta

```
<div class="card-header row">
```

Define la cabecera de la tarjeta.

La clase `row` permite distribuir el título a un lado y los botones al otro.

Bloque de título dinámico

```
<div>
  <h2 id="title">Alerta #-</h2>
  <p id="subtitle" class="muted">-</p>
</div>
```

Este bloque muestra el título y subtítulo de la alerta.

El valor inicial es un placeholder:

```
Alerta #-
-
```

Después, `alert_detail.js` lo actualiza con datos reales.

Título dinámico

```
<h2 id="title">Alerta #</h2>
```

El `id`:

```
id="title"
```

permite a JavaScript modificar el texto.

En `alert_detail.js`:

```
qs("title").textContent = `Alerta #${a.id}`;
```

Por tanto, si la alerta tiene ID 5, se mostrará:

```
Alerta #5
```

Subtítulo dinámico

```
<p id="subtitle" class="muted"></p>
```

Muestra información resumida de la alerta.

En `alert_detail.js` se actualiza así:

```
qs("subtitle").textContent = `status=${a.status} · group_key=${a.group_key ?? "-"}`;
```

Ejemplo:

```
status=open · group_key=server-01
```

Contenedor de acciones de estado

```
<div class="actions">
```

Agrupar los botones que permiten cambiar el estado de la alerta.

Botón ACK

```
<button id="btnAck" class="btn primary" type="button">ACK</button>
```

Botón para reconocer una alerta.

Desglose:

```
id="btnAck"
```

Identificador usado por JavaScript.

```
class="btn primary"
```

Aplica estilo de botón principal.

```
type="button"
```

Evita comportamiento de envío de formulario.

```
ACK
```

Texto visible.

Funcionalmente, este botón cambia el estado a:

```
ack
```

Botón CLOSE

```
<button id="btnClose" class="btn danger" type="button">CLOSE</button>
```

Botón para cerrar una alerta.

Desglose:

```
id="btnClose"
```

Identificador usado por JavaScript.

```
class="btn danger"
```

Aplica estilo de acción peligrosa o crítica.

```
CLOSE
```

Texto visible.

Funcionalmente, este botón cambia el estado a:

```
closed
```

Botón REOPEN

```
<button id="btnReopen" class="btn" type="button">REOPEN</button>
```

Botón para reabrir una alerta.

Funcionalmente, cambia el estado a:

```
open
```

Esto permite recuperar una alerta cerrada o reconocida.

Cierre de cabecera de tarjeta

```
</div>  
</div>
```

Cierra acciones y cabecera de tarjeta.

Caja de error

```
<div id="errorBox" class="alert error hidden"></div>
```

Elemento reservado para mostrar errores.

Inicialmente está oculto mediante la clase:

```
hidden
```

Se muestra desde JavaScript usando:

```
show(qs("errorBox"), "mensaje")
```

Caja de información

```
<div id="infoBox" class="alert info hidden"></div>
```

Elemento reservado para mostrar mensajes informativos.

Por ejemplo, cuando se actualiza correctamente el estado de una alerta.

En `alert_detail.js`:

```
show(info, `Estado actualizado a: ${a.status}`);
```

Grid principal

```
<div class="grid">
```

Crea una estructura en dos columnas.

Dentro contiene:

1. Bloque key-value con datos principales.
2. Panel con título y notas.

En CSS, `.grid` define un diseño con dos columnas en pantallas grandes y una columna en pantallas pequeñas.

Bloque key-value

```
<div class="kv">
```

Crea un bloque de pares clave-valor.

Aquí se muestran datos técnicos de la alerta.

La clase `kv` aplica un diseño de dos columnas:

```
clave → valor
```

Campo `id`

```
<div class="k">id</div><div class="v" id="v_id">--</div>
```

Muestra el identificador de la alerta.

La clave es:

```
id
```

El valor tiene:

```
id="v_id"
```

JavaScript lo actualiza así:

```
qs("v_id").textContent = a.id;
```

Campo `status`

```
<div class="k">status</div><div class="v"><span id="v_status" class="badge"></span></div>
```

Muestra el estado de la alerta.

El valor está dentro de un `span` con clase `badge`.

Esto permite mostrar el estado con formato visual diferenciado.

JavaScript actualiza:

```
qs("v_status").textContent = a.status;  
qs("v_status").className = statusBadgeClass(a.status);
```

Campo `group_key`

```
<div class="k">group_key</div><div class="v" id="v_group_key"></div>
```

Muestra la clave de agrupación de la alerta.

Ejemplo:

```
server-01
```

Si no existe, JavaScript muestra:

```
-
```

Campo `rule_id`

```
<div class="k">rule_id</div><div class="v" id="v_rule_id"></div>
```

Muestra el ID de la regla que generó la alerta.

Este valor conecta la alerta con la tabla `rules`.

Campo `event_id`

```
<div class="k">event_id</div><div class="v" id="v_event_id"></div>
```

Muestra el ID del evento que disparó la alerta.

Este valor conecta la alerta con la tabla `events`.

Campo `created_at`

```
<div class="k">created_at</div><div class="v" id="v_created_at"></div>
```

Muestra la fecha de creación de la alerta.

JavaScript formatea la fecha con:

```
fmtDate(a.created_at)
```

Campo `updated_at`

```
<div class="k">updated_at</div><div class="v" id="v_updated_at"></div>
```

Muestra la fecha de última actualización.

Este campo cambia cuando se actualiza el estado de la alerta.

Ejemplo:

```
open → ack  
ack → closed
```

Cierre del bloque `kv`

```
</div>
```

Cierra el bloque de datos clave-valor.

Panel lateral

```
<div class="panel">
```

Crea un segundo bloque visual dentro del grid.

Contiene el título de la alerta y notas explicativas.

Título del panel

```
<h3>Título</h3>
```

Subtítulo interno del panel.

Valor del título de la alerta

```
<p id="v_title" class="mono"></p>
```

Muestra el título real de la alerta.

El `id`:

```
id="v_title"
```

permite actualizarlo desde JavaScript.

La clase:

```
class="mono"
```

usa fuente monoespaciada.

En `alert_detail.js`:

```
qs("v_title").textContent = a.title;
```

Apartado de notas

```
<h3>Notas</h3>
```

Introduce una lista de notas técnicas.

Lista de notas

```
<ul class="muted">
```

Crea una lista con estilo de texto secundario.

Nota sobre estado

```
<li>Este MVP gestiona solo el estado (<code>open|ack|closed</code>).</li>
```

Indica que en esta versión solo se modifica el estado de la alerta.

No se editan otros campos como título, regla o evento asociado.

Nota sobre evento asociado

```
<li>Los eventos asociados (detalle del evento) no están en <code>Alert0ut</code>; se integrarían usando <code>/events</code> en una iteración posterior.</li>
```

Esta nota es importante.

Indica una limitación del frontend actual.

La vista usa `Alert0ut`, que no incluye información detallada del evento.

Por eso se muestran:

```
rule_id
event_id
```

pero no datos enriquecidos como:

```
event_source
event_severity
event_message
```

Punto técnico: en el backend ya existe `AlertUI0ut` y el endpoint:

```
GET /alerts/{alert_id}/ui
```

que permitiría mejorar esta vista sin consultar `/events` por separado.

Pero el comentario refleja el estado actual del frontend.

Cierre de panel, grid y tarjeta

```
</ul>
</div>
</div>
</section>
```

Cierra la lista, el panel, el grid y la tarjeta principal.

Cierre del contenido principal

```
</main>
```

Cierra el bloque principal.

Carga de `app.js`

```
<script src="./assets/app.js"></script>
```

Carga funciones comunes del frontend.

Este script define funciones como:

```
qs()  
show()  
hide()  
fmtDate()  
statusBadgeClass()  
apiFetch()  
getQueryParam()
```

Debe cargarse antes de `alert_detail.js`.

Carga de `alert_detail.js`

```
<script src="./assets/alert_detail.js"></script>
```

Carga la lógica específica de esta página.

Este archivo:

- Lee el parámetro `id` de la URL.
 - Consulta la alerta con `GET /alerts/{id}`.
 - Renderiza los datos en los campos HTML.
 - Gestiona los botones `ACK`, `CLOSE` y `REOPEN`.
 - Actualiza el estado con `PATCH /alerts/{id}`.
-

Cierre del documento

```
</body>  
</html>
```

Cierra el cuerpo y el documento HTML.

Relación con el flujo técnico del laboratorio

`alert.html` representa la vista individual de una alerta.

La relación técnica es:

```
index.html  
  ↓  
usuario pulsa Ver  
  ↓  
alert.html?id=X  
  ↓
```

```
alert_detail.js lee id
↓
GET /alerts/X
↓
renderAlert()
↓
usuario pulsa ACK / CLOSE / REOPEN
↓
PATCH /alerts/X
↓
backend actualiza Alert.status
↓
frontend refresca datos visibles
```

Esta página es el punto donde el usuario interactúa directamente con el ciclo de vida de la alerta.

8 Relación con el flujo SOC

La página permite representar un flujo SOC básico:

```
open
  ↓ ACK
ack
  ↓ CLOSE
closed
```

También permite reabrir:

```
ack → open
closed → open
```

Los botones disponibles dependen del estado actual.

Esta lógica no está en el HTML, sino en:

```
frontend/assets/alert_detail.js
```

El HTML solo define los botones y los puntos donde se mostrarán los datos.

9 Errores típicos o puntos importantes

La página necesita parámetro `id`

La URL debe tener este formato:

```
alert.html?id=1
```

Si falta el parámetro `id`, `alert_detail.js` muestra un error.

`alert.html` no contiene datos reales inicialmente

Los campos empiezan con:

```
-
```

Después JavaScript los reemplaza con datos de la API.

Los IDs del HTML son críticos

Elementos como:

```
title
subtitle
btnAck
btnClose
btnReopen
errorBox
infoBox
v_id
v_status
v_group_key
v_rule_id
v_event_id
v_created_at
v_updated_at
v_title
```

son usados por `alert_detail.js`.

Si se cambia un ID en HTML, hay que modificar también el JavaScript.

El orden de scripts importa

Primero se carga:

```
<script src="./assets/app.js"></script>
```

Después:

```
<script src="./assets/alert_detail.js"></script>
```

Si se invierte el orden, `alert_detail.js` podría fallar porque depende de funciones definidas en `app.js`.

La vista usa alerta básica

La nota interna indica que la página trabaja con `AlertOut`.

Por eso muestra `rule_id` y `event_id`, pero no contexto enriquecido del evento.

Para mejorarla, se podría consumir:

```
GET /alerts/{alert_id}/ui
```

Botones sin formulario

Los botones tienen:

```
type="button"
```

Esto evita que intenten enviar un formulario inexistente.

10 Posible mejora futura

Actualmente, el detalle consulta:

```
GET /alerts/{alert_id}
```

Esto devuelve `AlertOut`.

Una mejora directa sería cambiarlo a:

```
GET /alerts/{alert_id}/ui
```

Así la página podría mostrar también:

```
rule_name
event_ts
event_source
event_severity
event_message
```

Con esa mejora, el panel de notas podría transformarse en un panel de contexto real del evento.

11 Comandos útiles relacionados

Servir el frontend:

```
cd ~/siem-lab/frontend
python3 -m http.server 5173
```

Abrir una alerta concreta:

```
http://localhost:5173/alert.html?id=1
```

Probar endpoint básico que usa la página:

```
curl http://localhost:8000/alerts/1
```

Probar endpoint enriquecido disponible en backend:

```
curl http://localhost:8000/alerts/1/ui
```

Actualizar alerta a `ack` :

```
curl -X PATCH http://localhost:8000/alerts/1 \
-H "Content-Type: application/json" \
-d '{
  "status": "ack"
}'
```

Actualizar alerta a `closed` :

```
curl -X PATCH http://localhost:8000/alerts/1 \
-H "Content-Type: application/json" \
-d '{
  "status": "closed"
}'
```

Reabrir alerta:

```
curl -X PATCH http://localhost:8000/alerts/1 \
-H "Content-Type: application/json" \
-d '{
  "status": "open"
}'
```

Abrir Swagger:

```
http://localhost:8000/docs
```